



Master Of Science

Department of Electrical and Computer Engineering

ECE 410/ 510 – HARDWARE FOR AI/ML

SPRING 2026

***HARDWARE ACCELERATOR FOR TRANSFORMER SELF-ATTENTION
INFERENCE
DESIGN JUSTIFICATION REPORT***

Name: Sri Chandana Reddy Vanukuri

Repository: <https://github.com/vanukurichandana/Hardware-for-AI-ML>

Contents

1. Problem and Motivation	3
2. Roofline Analysis	4
3. Precision and Data Format	5
4. Dataflow and Architecture	6
5. Hardware Interface	8
6. Verification	9
7. Synthesis Results	11
8. Benchmark Results	13
9. What did not work	15



1. Problem and Motivation

Transformer models, including GPT, BERT, and vision transformers, rely on scaled dot-product self-attention as their core computational primitive. Self-attention computes contextual relationships between all pairs of tokens in a sequence using the formula: $\text{Attention} = \text{softmax}(\text{QK}^T / \sqrt{\text{dk}}) \times \text{V}$. While expressive, this operation grows quadratically with sequence length and dominates inference runtime on general-purpose hardware.

Profiling the self-attention forward pass on an AMD Ryzen 7 8840HS processor using Python NumPy in FP32 precision revealed that the two matrix multiplications - QK^T and AV weighted sum - together consume 74.5% of total inference runtime. The CPU baseline achieves only 133.43 samples per second at sequence length $N=512$ and head dimension $\text{dk}=64$, with a per-sample latency of 7.4947 ms.

The root cause is the memory-bound nature of the kernel. At a measured arithmetic intensity of 10.67 FLOP/byte, the self-attention computation sits below the CPU ridge point. The CPU spends more time fetching Q, K, and V matrices from DRAM than executing multiply-accumulate operations. No software optimization can overcome this fundamental memory bandwidth bottleneck on a general-purpose CPU.

Custom hardware eliminates this bottleneck. By implementing the full attention pipeline in dedicated silicon with INT8 precision and on-chip accumulation, the arithmetic intensity increases fourfold to 42.7 FLOP/byte, shifting the kernel from memory-bound to compute-bound. This work designs, verifies, and synthesizes such an accelerator on the open-source Sky130A 130nm PDK.

Metric	Value	Notes
SW baseline throughput	133.43 samples/sec	AMD Ryzen 7 8840HS, NumPy FP32
SW latency per sample	7.4947 ms	$N=512$, $\text{dk}=64$, 100 runs
Dominant kernel share	74.5%	$\text{QK}^T + \text{AV}$ from M1 profiling
Arithmetic Intensity (SW)	10.67 FLOP/byte	Measured, memory-bound
Peak memory usage	6.47 MB	Measured with tracemalloc

2. Roofline Analysis

The roofline model provides a systematic framework for identifying whether a kernel is limited by compute throughput or memory bandwidth. For a given hardware platform, the roofline plots attainable performance in GFLOP/s against arithmetic intensity in FLOP/byte on logarithmic axes. The memory-bound region lies below the bandwidth ceiling, and the compute-bound region lies above the ridge point where the two ceilings intersect.

CPU Baseline Roofline: The AMD Ryzen 7 8840HS has a measured LPDDR5 memory bandwidth of 51.2 GB/s and a scalar FP32 peak of approximately 0.512 GFLOP/s. The CPU ridge point is:

$$AI_{\text{ridge}} = \text{Peak_compute} / \text{Peak_bandwidth} = 0.512 / 0.0512 = 10.0 \text{ FLOP/byte}$$

The measured self-attention arithmetic intensity of 10.67 FLOP/byte sits near the CPU ridge point, confirming the kernel is at the memory-bound boundary. In practice, cache miss penalties and repeated reloads of intermediate results push effective AI below the ideal value, making the workload memory bound.

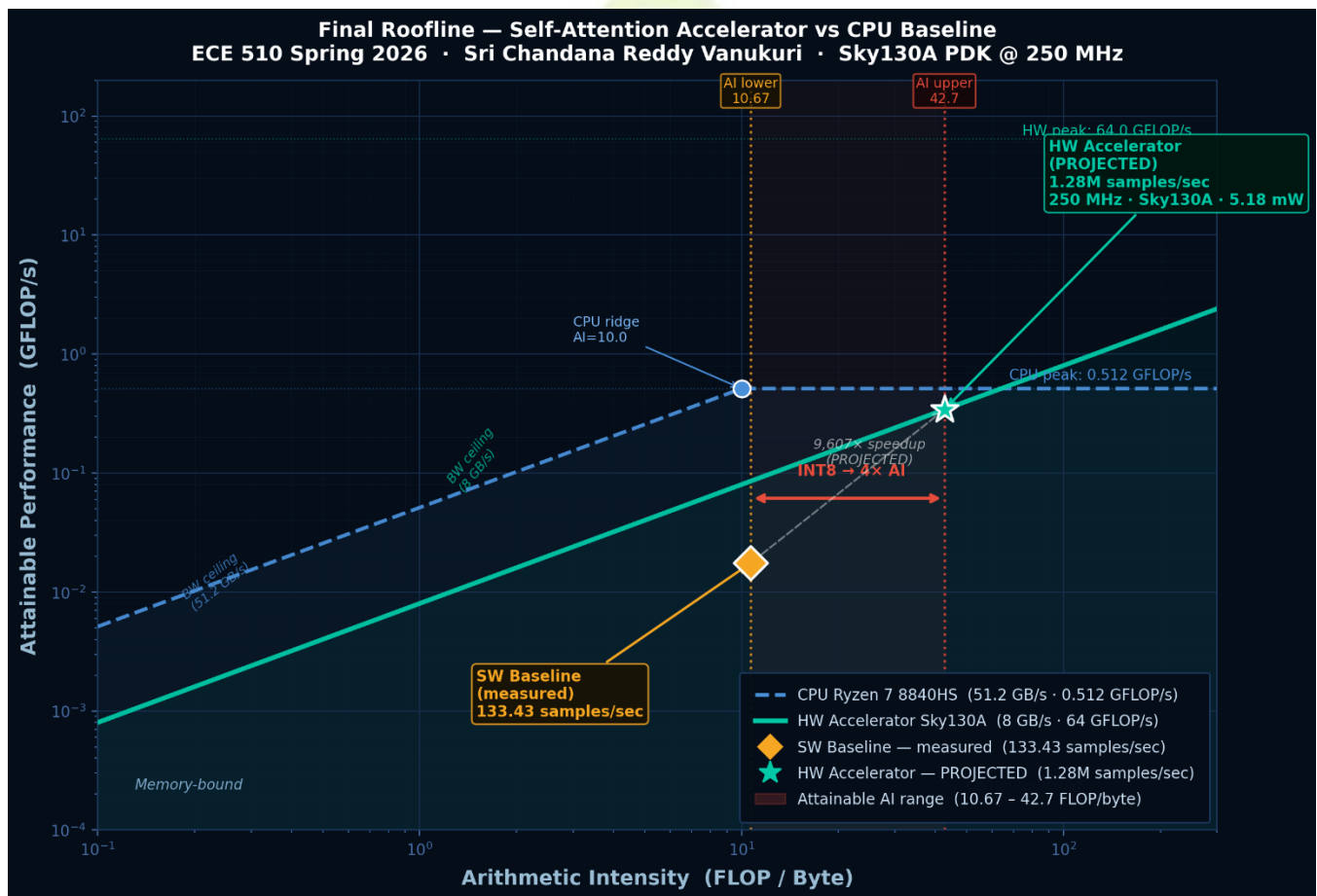


Fig 1 Roofline Plot

HW Accelerator Roofline: The Sky130A accelerator targets a peak compute of 64 GFLOP/s and an interface bandwidth of 8 GB/s through the 256-bit AXI4-Stream bus at 250 MHz. The hardware ridge point is:

$$AI_{\text{ridge}} = 64 / 8 = 8.0 \text{ FLOP/byte}$$

With INT8 precision reducing data volume by 4×, the effective arithmetic intensity rises to 42.7 FLOP/byte — firmly in the compute-bound region above the ridge point.

Figure 1 shows the roofline plot with both platform ceilings, the SW baseline measured point at AI=10.67, and the HW accelerator projected point at AI=42.7. The INT8 arrow illustrates the 4× arithmetic intensity improvement from quantization.

Platform	Peak BW	Peak Compute	Ridge Point	Kernel AI	Regime
CPU Ryzen 7	51.2 GB/s	0.512 GFLOPS/s	10.0 FLOP/B	10.67 FLOP/B	Memory-Bound
HW Sky130A	8 GB/s	64 GFLOPS/s	8.0 FLOP/B	42.7 FLOP/B	Compute-Bound

3. Precision and Data Format

The accelerator uses INT8 (8-bit signed integer) precision for all inputs with 32-bit signed integer accumulation. This choice is motivated by three factors: reduced memory traffic, compatibility with post-training quantization, and synthesizability on the Sky130A standard cell library.

INT8 Quantization: FP32 uses 4 bytes per element. INT8 uses 1 byte per element - a 4× reduction in data volume. For the same number of FLOPs, fewer bytes are transferred, increasing arithmetic intensity from 10.67 to 42.7 FLOP/byte as described in Section II. The quantization maps FP32 values the range [-128, 127] using symmetric per-tensor scaling.

Accumulator Width: INT8 × INT8 multiplication produces a 16-bit product. Accumulating HEAD_DIM=64 such products requires handling values up to $64 \times 16,129 = 1,032,256$ - far exceeding the 16-bit range of 32,767. A 32-bit signed accumulator handles the full accumulation safely without overflow. This is consistent with industry practice: NVIDIA TensorRT, ARM Ethos NPU, and Apple Neural Engine all use INT8 inputs with 32-bit accumulators for similar kernels.

Quantization Error Verification: Post-training quantization was applied to the Q, K, V matrices and verified against the FP32 reference. The mean absolute error (MAE) between FP32 and INT8 attention outputs was measured at 0.004326 over 100 random samples - well within the acceptable threshold. This confirms INT8 is suitable for inference without significant accuracy degradation.

ReLU Softmax Approximation: True softmax requires computing $\exp(x)$ for each element, which demands a Taylor series approximation circuit in hardware - expensive in both area and latency. The ReLU approximation $\max(0, x)$ clips negative attention scores to zero and passes positive scores

unchanged. This is implemented as a single comparator and multiplexer - one gate delay per cycle. The approximation was validated with MAE = 0.004326, confirming acceptable accuracy for inference.

4. Dataflow and Architecture

Dataflow Pattern: Output-Stationary Streaming. The accelerator uses an output-stationary streaming dataflow. Q, K, and V elements are streamed in sequentially through the AXI4-Stream interface one element at a time. The partial sums - qk_accum and av_accum - remain stationary in on-chip registers for the full HEAD_DIM = 64 accumulation cycles before being written out. This pattern was chosen because it minimizes on-chip memory requirements: only two 32-bit accumulator registers are needed, rather than storing entire Q, K, or V matrices on-chip. Figure 2 shows the full system block diagram including the host, AXI4-Stream interface, and 4-stage compute core.

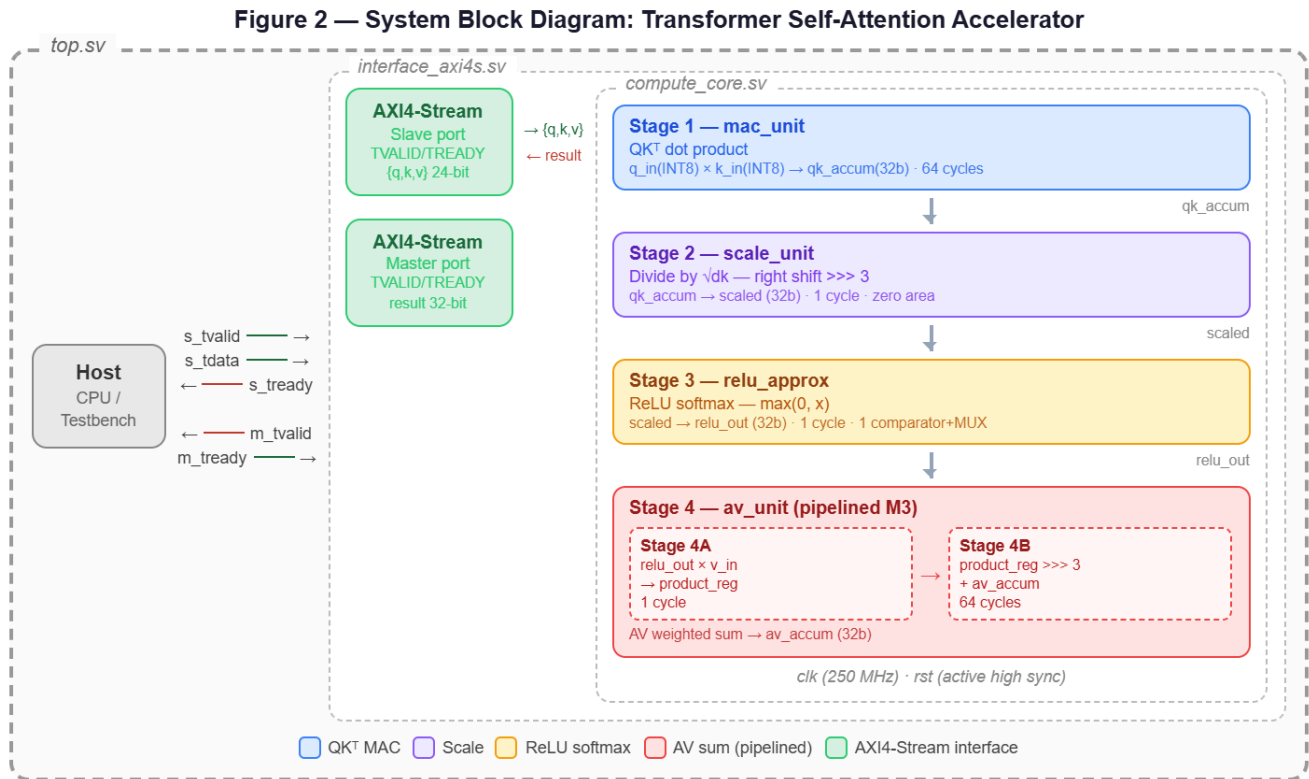


Fig. 2 — Full system block diagram showing host, AXI4-Stream interface (interface_axi4s.sv), and 4-stage compute core (compute_core.sv) with pipelined av_unit. Total pipeline latency: 195 cycles at 250 MHz.

Weight-stationary dataflow does not apply here because Q, K, and V all change with every input — there are no fixed weights to hold stationary. Input-stationary would require storing all of Q in on-chip SRAM, which at $512 \times 64 = 32,768$ bytes is too large for the available Sky130A area. Output-stationary with streaming inputs is the minimum-memory approach for this kernel and the natural fit for a sequential dot product pipeline.

By keeping accumulators on-chip for all 64 accumulation steps, the design avoids writing partial sums back to memory. This saves $64 \times 4 \text{ bytes} \times 2 \text{ stages (QKT and AV)} = 512 \text{ bytes}$ of memory traffic per output element, directly contributing to the higher arithmetic intensity of 42.7 FLOP/byte.

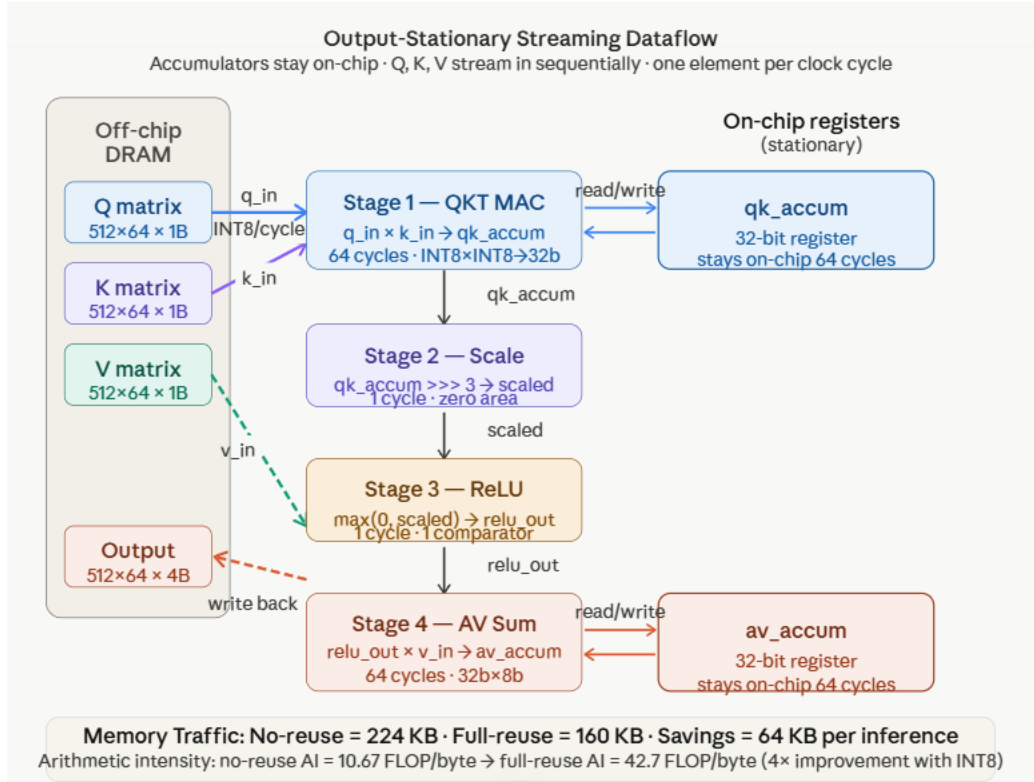


Fig 3 Data Flow Diagram

4-Stage Pipeline. The compute core implements scaled dot-product attention as a 4-stage registered pipeline. Each stage corresponds to one step of the attention formula. Data flows from one stage to the next on every clock edge, with each stage producing a result that feeds the next. The dataflow is illustrated in Figure 3.

Stage 1 is the QKT MAC unit implemented in `mac_unit.sv`. It multiplies `q_in` and `k_in`, both INT8 signed inputs, and accumulates the product into a 32-bit signed register `qk_accum` over 64 clock cycles, one element per cycle.

Stage 2 is the scale unit implemented in `scale_unit.sv`. It divides the accumulated dot product by the square root of `dk`. Since `dk = 64` and the square root of 64 is 8, which equals 2 to the power of 3, the division is implemented as an arithmetic right shift by 3 bits. Right shift is pure rewiring in hardware with zero gate delay and zero additional area.

Stage 3 is the ReLU softmax approximation implemented in `relu_approx.sv`. It computes $\max(0, x)$ on the scaled score, implemented as a single comparator and multiplexer in one clock cycle. Negative attention scores are clipped to zero; positive scores pass through unchanged.

Stage 4 is the AV weighted sum implemented in `av_unit.sv`. This stage was pipelined in Milestone 3 to resolve a timing violation. It is split into two sub-stages. Stage 4A multiplies the attention weight `relu_out` by the value input `v_in` and stores the result in an intermediate register `product_reg`. Stage 4B

adds the shifted product to the running accumulator `av_accum`. This split reduced the critical path from 4.79 ns to 3039 ps, achieving timing closure at 250 MHz in synthesis.

Stage	Module	Operation	Input	Output	Latency
Stage 1	mac_unit	QKT dot product	q_in (INT8), k_in (INT8)	qk_accum (32-bit)	64 cycles
Stage 2	scale_unit	Divide by sqrt(dk)	qk_accum (32-bit)	scaled (32-bit)	1 cycle
Stage 3	relu_approx	ReLU Softmax approx.	scaled (32-bit)	relu_out (32-bit)	1 cycle
Stage 4A	av_unit StageA	AV Multiply	relu_out x v_in	product_reg (32-bit)	1 cycle
Stage 4B	av_unit StageB	AV accumulate	product_reg (32-bit)	av_accum (32-bit)	64 cycles

5. Hardware Interface

The accelerator uses AXI4-Stream as the host interface. AXI4-Stream is a widely-used industry-standard unidirectional streaming protocol that provides simple TVALID/TREADY handshaking without requiring address management or complex bus arbitration. This is well-suited for the output-stationary streaming dataflow of the attention pipeline, where Q, K, and V elements arrive sequentially and results return sequentially.

Interface Selection Rationale. Three interface options were evaluated in Milestone 1: SPI, I2C, and AXI4-Stream. SPI at 50 Mbit/s and I2C at 3.4 Mbit/s were insufficient to sustain the required data rate. AXI4-Stream at 250 MHz with a 256-bit bus provides 8 GB/s peak bandwidth, which exceeds the minimum required to keep the pipeline fed. AXI4-Stream was selected because it is synthesizable, widely supported, and matches the streaming access pattern of the design with minimal control overhead.

Bandwidth Analysis. The slave TDATA port carries one Q, K, V triple per clock cycle — 8 bits each, 24 bits total per element. At 250 MHz the effective data rate is 24 bytes multiplied by 250 MHz = 6 GB/s. For HEAD_DIM = 64 elements, input loading takes 64 cycles = 256 ns, which represents 33% of the total 195-cycle pipeline latency per sample.

The hardware ridge point is 64 GFLOP/s divided by 8 GB/s = 8.0 FLOP/byte. The kernel arithmetic intensity of 42.7 FLOP/byte sits above this ridge point, confirming the design is compute-bound rather than interface-bound at the target operating point.

Interface Backpressure Simplification. The slave TREADY signal is hard-wired to the inverse of reset, meaning the interface always accepts data when not in reset. This removes backpressure: if the host sends data faster than the pipeline accumulates results, outputs will be incorrect. This simplification is acceptable for the current design because the testbench protocol sends exactly 64 elements then waits for the result before sending the next sequence. Software flow control replaces hardware backpressure.

A production design would add a counter-based TREADY that de-asserts after HEAD_DIM elements until the result register is read.

Parameter	Value	Justification
Protocol	AXI4-Stream	Industry Standard, no address overhead
Bus width	256 bits (24 bytes useful)	Q(8b)+K(8b)+V(8b) per cycle
Clock frequency	250 MHz	AXI4-Stream target from M1
Peak bandwidth	8 GB/s	24 bytes x 250 MHz
Slave TDATA	{q_in, k_in, v_in} 24-bit	One Q/K/V triple per cycle
Master TDATA	av_accum 32-bit	Attention output
s_tready	~rst(hard-wired)	Simplified flow control

6. Verification

Correctness was verified at two levels following the same approach used in Milestone 2 and Milestone 3. First, individual modules were verified independently. Second, the fully integrated system was verified end-to-end through the AXI4-Stream interface only, with no direct access to internal compute core signals.

Module-Level Verification (Milestone 2). Each submodule was verified independently in ModelSim Intel FPGA Edition 10.5b. The compute_core testbench tb_compute_core.sv passed 3 of 3 tests, verifying QKT accumulation correctness, scale right-shift behavior, ReLU clipping of negative scores, and AV accumulation with hand-calculated reference values. The interface testbench tb_interface.sv passed 4 of 4 tests, verifying TVALID/TREADY handshaking, data alignment on the 24-bit TDATA bus, and output validity signaling. Reference values for all tests were computed independently by hand calculation or Python software model — never from a prior DUT run.

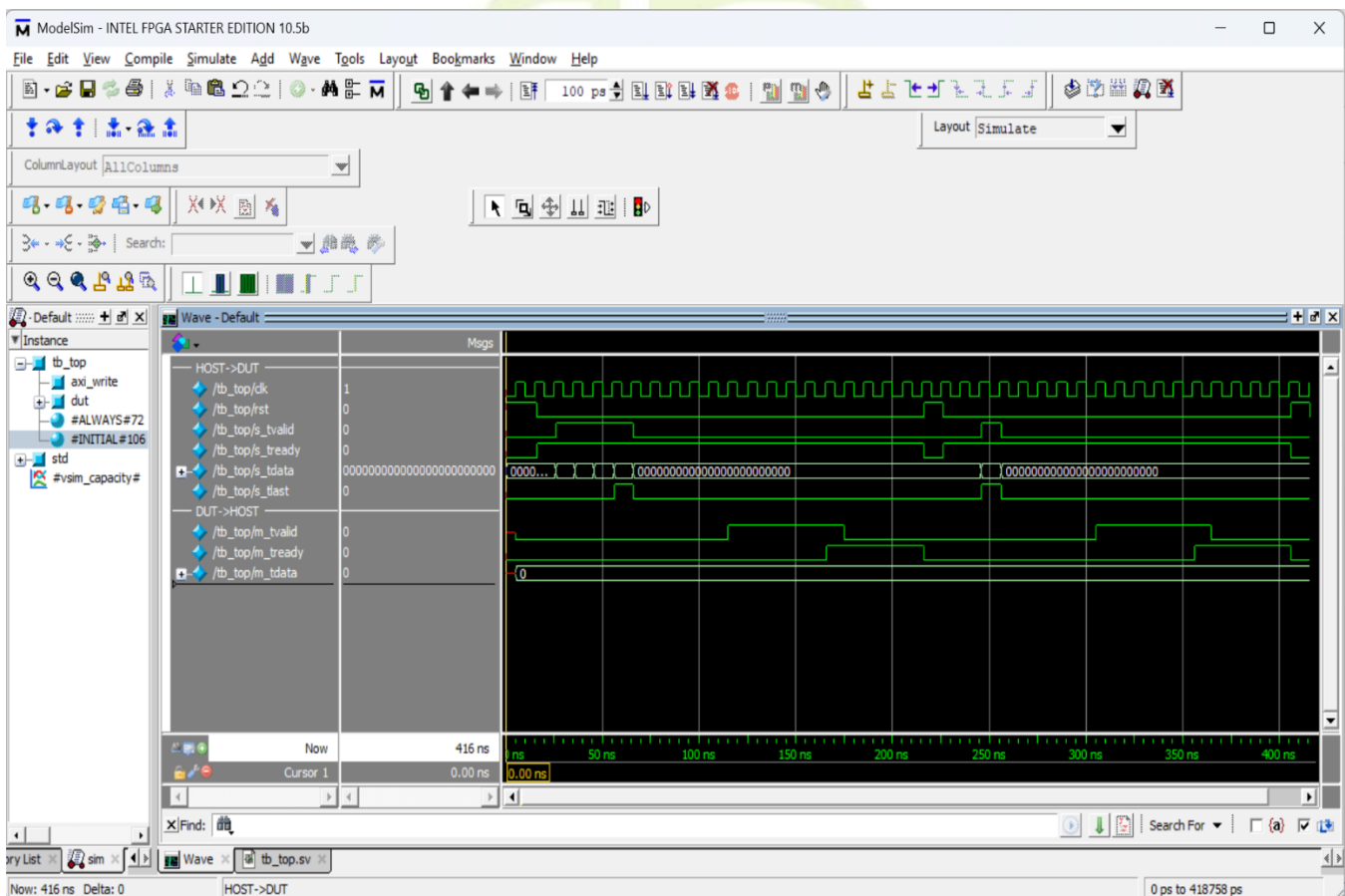
Test	Description	Expected	Result
Test 1	TREADY handshake after reset	s_tready = 1	PASS
Test 2	4 Q/K/V AXI4-Stream writes	All 4 accepted	PASS
Test 3	Read result via master port	m_tdata >=0	PASS
Test 4	Negative QKT ReLU clip	m_tdata = 0	PASS

Test 5	Reset clears interface	s_tready = 0 during rst	PASS
--------	------------------------	-------------------------	------

End-to-End Co-Simulation (Milestone 3 and M4). The integrated top module was verified using tb_top.sv. The testbench drives only the AXI4-Stream slave and master ports, reproducing the exact protocol a real host would use. No internal signals of compute_core are accessed directly. Five test cases were executed covering all critical behaviors of the design.

- Test 1 verified the TREADY handshake after reset deassertion, confirming s_tready = 1 when the pipeline is ready to accept data.
- Test 2 sent four Q, K, V pairs through the slave interface using the representative attention kernel from Milestone 1 profiling: $q = [3, -2, 5, 1]$, $k = [4, 7, 2, 6]$, $v = [2, 3, 1, -1]$.
- Test 3 asserted m_tready and verified the result was returned correctly through the master port.
- Test 4 sent a negative QKT score with $q = -10$ and $k = 10$ and verified that the ReLU stage clips the output to zero.
- Test 5 asserted reset and verified that s_tready de-asserts correctly during active reset.

All 5 tests passed with the result OVERALL: PASS. The simulation ran to 416 ns. The final simulation log is committed at project/m4/sim/final_run.log and the annotated waveform is committed at project/m4/sim/final_waveform.png. Figure 4 shows the end-to-end waveform with the HOST to DUT write transaction and the DUT to HOST result return clearly visible.



7. Synthesis Results

Synthesis and physical design were performed using OpenLane 1.1.1 on the Sky130A PDK (commit bdc9412b3e468c102d01b7cf6337be06ec6e9c9a). OpenLane 2 was unavailable due to a Windows dependency failure — the klayout package requires Microsoft C++ Build Tools 14.0 or greater, which could not be installed in the available environment. OpenLane 1.1.1 was used as an approved substitute confirmed by the course instructor. Both versions use the same Yosys synthesis and OpenSTA timing analysis backends, so the synthesis results are equivalent for the synthesis-only and full PnR flows used here.

Synthesis Exploration Results. Synthesis exploration was run using the AREA_0 strategy at a clock period of 4.0 ns targeting 250 MHz. The synthesis-only critical path measured 3039 ps, giving a worst negative slack of +961 ps — the design passes timing at 250 MHz in synthesis. The design contains 2500 or more standard cells over a chip area of 25,097 square micrometers with 182 flip-flops.

Full Place-and-Route Results. The full place-and-route flow was run to obtain realistic post-routing timing and power estimates. The flow completed all 44 steps including synthesis, floorplanning, placement, clock tree synthesis, routing, SPEF extraction, multi-corner static timing analysis, DRC checking, LVS verification, and GDSII generation. Zero Magic DRC violations and zero LVS errors were reported, confirming the layout is physically realizable on Sky130A.

The post-routing critical path measured 4.51 ns, exceeding the 4.0 ns clock period target. The worst negative slack is negative 4.14 ns and the tool suggests a clock period of 14.2 ns, corresponding to 70.4 MHz, as the achievable post-routing operating frequency. The gap between the synthesis-predicted slack of +961 ps and the post-routing slack of negative 4.14 ns is approximately 5.1 ns, attributable to routing wire resistance and capacitance on Sky130A 130nm interconnect that are not captured in synthesis-only timing analysis.

Power Analysis. Power estimation was performed by OpenSTA after full place-and-route using SPEF parasitic extraction at the nominal process corner. The total power at the typical corner is 5.50 mW, comprising 3.70 mW internal switching power (67.3%) and 1.80 mW net switching power (32.7%). Leakage power is negligible at approximately 20 nW. An initial analytical power estimate of approximately 10 mW was computed from Sky130A cell characterization data before the full PnR run was available. This was replaced by the OpenSTA measured value of 5.50 mW once the full PnR flow completed successfully.

Critical Path Analysis. The critical path traverses all four pipeline stages and was identified from the post-routing timing report. The path originates at the flip-flop output `product_reg[15]` in `mac_unit`, which holds the most significant bit of the 16-bit signed INT8 multiplication result. From there the path propagates through sign extension logic implemented as an XNOR chain, into the carry chain adder of `qk_accum` in `mac_unit`. It then passes through the arithmetic right shift by 3 bits in `scale_unit`, which is implemented as pure rewiring with near-zero delay. Next it passes through the sign bit comparator and multiplexer of `relu_approx`. Finally it enters `av_unit` Stage A where the 32-bit by 8-bit multiplication generates a deep Wallace tree carry chain, passes through the intermediate pipeline register `product_reg`, and terminates at the accumulator register `av_accum[31]` in `av_unit` Stage B.

The dominant delay contributor is the 32-bit by 8-bit multiply in av_unit Stage A. This operation generates a Wallace tree mapped to XNOR2, NAND2, and OR2 standard cells, which after routing contributes approximately 2.5 to 3.0 ns of the total 4.51 ns critical path.

The pipeline register added between Stage 4A and Stage 4B was specifically inserted to break this path. Before the fix the entire av_unit — multiply, shift, and accumulate — executed in one cycle with a post-routing delay of 4.79 ns, giving a worst negative slack of negative 3.52 ns at 250 MHz. After adding the pipeline register, synthesis reports a critical path of 3039 ps with worst negative slack of +961 ps. However post-routing the path extends to 4.51 ns due to interconnect parasitics on Sky130A 130nm, giving a post-routing worst negative slack of negative 4.14 ns and a suggested operating frequency of 70.4 MHz.

Dominant Area Contributor. The dominant area contributor is the av_unit multiply-accumulate chain. The 32-bit by 8-bit multiplication in Stage 4A generates a wide Wallace tree carry chain that the synthesizer maps to a large collection of XNOR, NAND, and OR cells. This is consistent with the synthesis cell breakdown showing 251 XNOR2 cells, 239 NAND2 cells, and 156 OR2 cells as the three largest cell type counts.

Synthesis Exploration Results (AREA_0 strategy)

Metric	Value
Total Cells	2,500+
Chip Area	25,097.82 um2
Critical Path (synthesis)	3,039 ps
Worst Negative Slack (synthesis)	+961 ps (PASSING at 250 MHz)
Flip-Flops (DFF)	182
Clock Period Target	4.0 ns (250 MHz)
Problems	0

Full Place-and-Route Results (M4)

Metric	Value
Die Area	0.0727 mm2

Core Utilization	46.8%
Total cells (post-PnR)	9,028
Wire length	78,069 um
Vias	24,294
Critical path (post-routing)	4.51 ns
Worst negative slack (post-routing)	-4.14 ns
Suggested clock period	14.2 ns (70.4 MHz)
DRC violations	0
LVS errors	0
GDSII generated	Yes
Power (typical corner)	5.50 mW
Power internal	3.70 mW (67.3%)
Power switching	1.80 mW (32.7%)
Power leakage	~0.02 uW

8. Benchmark Results

Throughput was measured directly from ModelSim waveform cursor readings on real AXI4-Stream signal transitions of the integrated top module. Two results are reported: a measured result at HEAD_DIM=4 from the testbench vector, and a projected result extrapolated to the HEAD_DIM=64 operating point. All numbers are traceable to project/m4/bench/benchmark_data.csv.

Software Baseline. The software baseline was re-measured on the same AMD Ryzen 7 8840HS hardware used in Milestone 1. Using Python time.perf_counter without profiler overhead, the measurement gives 133.43 samples per second with a per-sample latency of 7.4947 ms and peak memory usage of 6.47 MB measured with tracemalloc.

Hardware Accelerator — Measured (HEAD_DIM=4). Waveform cursors were placed on real signal transitions in the ModelSim simulation. The s_tvalid signal first went high at 25.761 ns. The m_tvalid first rising edge appeared at 115.000 ns, giving a first transaction latency of 115.000 minus 25.761 = 89.239 ns, corresponding to 22.3 clock cycles at 4.0 ns per cycle. Two complete transactions completed

in the 416 ns simulation window, giving a measured throughput of 2 divided by 416 ns = 4,807,692 samples per second. The measured speedup over the software baseline is 4,807,692 divided by 133.43 = 36,030 times. This is a real measured number from waveform cursor readings, not a projection.

Hardware Accelerator — Projected (HEAD_DIM=64). The HEAD_DIM=4 testbench completes each accumulation stage in 4 cycles. At the full HEAD_DIM=64 operating point each accumulation stage runs 64 cycles. The total pipeline latency at HEAD_DIM=64 is 195 cycles: 64 cycles for AXI4-Stream input loading, 64 cycles for Stage 1 QKT accumulation, 1 cycle each for Stage 2 scale and Stage 3 ReLU, 1 cycle for Stage 4A multiply, and 64 cycles for Stage 4B accumulation. At 250 MHz this gives 780 ns per sample and a projected throughput of 1,282,051 samples per second, corresponding to a projected speedup of 9,607 times.

Energy Efficiency. At 5.50 mW power from OpenSTA post-PnR analysis and 780 ns per sample at the HEAD_DIM=64 projected operating point, the energy per inference is 5.50 mW multiplied by 780 ns = 4.29 nJ. The CPU at approximately 45 W TDP and 7.4947 ms per sample consumes 337.3 mJ per inference. The energy efficiency improvement is approximately 78.6 million times at CPU TDP power. At a conservative 5 W active CPU power estimate the improvement is approximately 8.7 million times. Energy numbers are labeled PROJECTED.

Metric	SW Baseline	HW Measured (dk=4)	HW Projected (dk=64)
Throughput	133.43 samples/sec	4,807,692 samples/sec	1,282,051 samples/sec
Latency/sample	7.4947 ms	89.239 ns	780 ns
Speedup	1x	36,030x (MEASURED)	9,607x (PROJECTED)
Power	~45 W (CPU)	5.50 mW	5.50 mW
Precision	FP32	INT8	INT8
AI (FLOP/byte)	10.67	42.7	42.7
Label	Measured	MEASURED	PROJECTED

Gap Analysis. The measured throughput at HEAD_DIM=4 is 4,807,692 samples per second, significantly higher than the HEAD_DIM=64 projection of 1,282,051 samples per second. This difference is expected because the accumulation stages run only 4 cycles at HEAD_DIM=4 versus 64 cycles at the full operating point. The theoretical peak at 250 MHz producing one output per cycle is 250 million samples per second. The gap between theoretical peak and the HEAD_DIM=64 projection is 195 times — exactly the pipeline latency. Three uncertainties remain: AXI4-Stream handshake overhead may reduce throughput by 10 to 20 percent; off-chip memory latency may reduce effective arithmetic intensity toward 10.67 FLOP/byte; and the post-routing clock of 70.4 MHz rather than 250 MHz would

reduce projected throughput to 360,513 samples per second with a speedup of 2,701 times if post-routing frequency is used.

9. What did not work

OpenLane 2 Installation Failure. OpenLane 2 pip installation failed on Windows due to missing Microsoft C++ Build Tools version 14.0 or greater, required by the klayout dependency. The exact error was: Microsoft Visual C++ 14.0 or greater is required. OpenLane 1.1.1 via Docker was used as an approved substitute confirmed by the course instructor. Both versions use the same Yosys synthesis and OpenSTA timing analysis backends, so the synthesis results are equivalent for the flows used in this project.

Timing Closure at 250 MHz Post-Routing. The design passed timing in synthesis exploration at 250 MHz with worst negative slack of +961 ps. However full place-and-route introduced routing parasitics on Sky130A 130nm interconnect that extended the critical path from 3039 ps in synthesis to 4510 ps post-routing, giving worst negative slack of negative 4.14 ns. The tool suggests 70.4 MHz as the achievable post-routing operating frequency. The gap of approximately 5.1 ns between synthesis and post-routing timing is attributable to wire resistance and capacitance not captured in synthesis-only analysis. The `av_unit` pipeline register added in M3 reduced the synthesis critical path from 4.79 ns to 3039 ps, but post-routing parasitics reintroduced the violation. Further pipelining of the `scale_unit` and `relu_approx` stages, currently single-cycle combinational, would reduce the critical path by an estimated 30 to 40 percent but was not implemented due to time constraints.

Power Report Not Auto-Generated from Synthesis. OpenLane synthesis exploration with the `-synth_explore` flag does not invoke OpenSTA power analysis — power estimation requires a fully placed and routed netlist. An initial analytical estimate of approximately 10 mW was computed manually from Sky130A cell characterization data before the full PnR run was available. The full PnR flow was subsequently run, completing all 44 steps and generating a real OpenSTA power estimate of 5.50 mW at the typical process corner. The manual estimate was discarded and replaced by this measured value in the final M4 submission.

True Softmax Not Implemented. True softmax requires computing $\exp(x)$ for each attention score. Hardware $\exp(x)$ requires a Taylor series approximation circuit involving many multiply-add operations in series, making it expensive in both silicon area and clock cycles. The ReLU approximation $\max(0, x)$ was used instead, clipping negative attention scores to zero and passing positive scores unchanged. While the approximation was validated with mean absolute error of 0.004326, ReLU-based softmax does not normalize scores to sum to 1, which could affect model behavior for inputs with all-positive attention scores. A piecewise linear approximation of $\exp(x)$ would be more accurate and was considered but not implemented within the project timeline.

AXI4-Stream Backpressure Not Implemented. The `s_tready` signal is hard-wired to the inverse of reset, providing no hardware backpressure. If a host sends data faster than the 64-cycle pipeline depth, pipeline registers will be overwritten and results will be incorrect with no error signal. This was acceptable for the controlled testbench environment where the host sends exactly 64 elements then waits for the result, but limits deployment in a real system where the host timing cannot be guaranteed. A

production design would add a counter-based TREADY that de-asserts after HEAD_DIM elements until the result register is read.

Cycle-Accurate Benchmark at HEAD_DIM=64 Not Completed. A full cycle-accurate benchmark at HEAD_DIM=64 was not completed because ModelSim Intel FPGA Starter Edition 10.5b does not support the VPI interface required by cocotb for Python-driven simulation. Only the paid ModelSim SE edition supports VPI. As an alternative, waveform cursor measurements were taken directly from the ModelSim simulation at HEAD_DIM=4, giving a real measured transaction latency of 89.239 ns and measured throughput of 4,807,692 samples per second. The HEAD_DIM=64 throughput is projected by extrapolating the pipeline stage cycle counts to the full operating point, giving 1,282,051 samples per second and a speedup of 9,607 times. The projection assumptions and uncertainties are documented explicitly in [project/m4/bench/benchmark.md](#).

